



порассуждай и предложи вариант как добавить в этот проект конвертации и элементы markdown

Я бы развивал ваш конвертер не как «кнопку Pandoc», а как небольшой локальный инструмент Docs-as-Code для ГОСТ-документации: Markdown остаётся единым исходником в GitHub, а приложение даёт автору визуальный, предсказуемый выпуск .docx по выбранному профилю оформления.

Ключевая идея: разделить в проекте семантику документа и оформление. В Markdown описывается содержание — разделы, требования, таблицы, замечания, рисунки, схемы; в профиле/шаблоне задаётся, как всё это выглядит в DOCX: Times New Roman 12, поля, колонтитулы, границы 0,5 pt, таблицы на ширину рабочей области. Pandoc поддерживает reference DOCX и пользовательские стили, которые можно назначать блокам и таблицам через custom-style.^[1] ^[2]

Целевая архитектура

Сейчас у вас уже есть хорошая основа:

```
Markdown-файл
  ↓
GUI на Python / Tkinter
  ↓
Pandoc + reference.docx
  ↓
DOCX рядом с исходником
```

Я бы расширил её до такой структуры:

```
md2gostdocx/
├── startmydocx.command
├── md_to_docx_gui.py
├── profiles/
│   ├── gost19-times12/
│   │   ├── reference.docx
│   │   ├── table_filter.py
│   │   └── profile.json
│   └── simple/
│       ├── reference.docx
│       └── profile.json
├── filters/
│   └── gost_docx.lua
└── examples/
```

```
| └─ example-tz.md
└─ output/
```

Где:

Компонент	Назначение
reference.docx	Стили Word: Times New Roman, заголовки, поля, колонтитулы, нумерация
profile.json	Настройки профиля: шрифт, размер, поля, таблицы, формат рисунков
gost_docx.lua	Семантическая обработка Markdown: классы таблиц, предупреждения, служебные блоки
table_filter.py	Финальная доводка DOCX: ширина таблиц, границы 0,5 pt, шрифт ячеек
GUI	Выбор источника, профиля, варианта сборки и просмотр журнала

Это даст возможность держать несколько профилей: например, «ГОСТ 19 / НГУ», «ГОСТ 34», «Отчёт НИР», «Презентационный DOCX», «Черновик без оформления».

Какие элементы Markdown добавить

Вместо сложной ручной стилизации в Word полезно договориться о небольшом «диалекте Markdown» для ТЗ. Он останется читаемым на GitHub и будет хорошо собираться в DOCX.

Структура документа

```
---
title: "Техническое задание"
document_code: "ТЗ-ИС-001-2026"
version: "1.0"
date: "06.09.2026"
customer: "НГУ"
developer: "Наименование исполнителя"
---

# 1 Общие положения

## 1.1 Наименование системы

Полное наименование: «Информационная система ...».

## 1.2 Основания для разработки

Разработка выполняется на основании договора № ...
```

Блок в начале между --- называется YAML-метаданными. Пока Pandoc сам не сделает полноценный ГОСТ-титул только из этих значений без шаблонной логики, но приложение может использовать их для:

- имени создаваемого DOCX;

- титульного листа;
- колонтитулов;
- номера версии;
- обозначения документа;
- журнала сборки;
- автоматической проверки заполненности обязательных полей.

Например, вместо имени ТЗ.docx автоматически получать:

```
ТЗ-ИС-001-2026_v1.0.docx
```

Нормальные заголовки

```
# 1 ОБЩИЕ ПОЛОЖЕНИЯ
## 1.1 Наименование программы
### 1.1.1 Полное наименование
```

Это наиболее стабильный вариант: Pandoc превращает их в Word-стили Heading 1-3, а Word может автоматически строить оглавление и многоуровневую нумерацию.

Избыточно не надо выделять такие строки ****жирным**** — это уже задача стиля заголовка в reference DOCX.

Таблица требований

```
## 3 Требования к системе
```

```
| № | Требование | Содержание требования |
| ---: | --- | --- |
| 1 | Производительность | Не менее 100 одновременных пользователей |
| 2 | Аутентификация | Интеграция с корпоративной учётной записью |
| 3 | Журналирование | Регистрация операций и ошибок |
```

Для конвертера можно добавить в интерфейс постоянное правило:

```
Все Markdown-таблицы:
- ширина: 100% доступной области;
- границы: чёрные, 0,5 pt;
- вертикальное выравнивание: по центру;
- шрифт: Times New Roman, 12 pt;
- шапка: полужирная, по центру;
- запрет разрыва строки шапки между страницами.
```

Именно этот слой я бы не пытался полностью удержать через Google Docs reference DOCX. Надёжнее после Pandoc обрабатывать готовый DOCX кодом: python-docx может пройти по всем таблицам и задать базовые свойства в OpenXML. Для действительно строгих правил

— ширина 100%, автоподбор, толщина всех линий — потребуется работа с XML Word, потому что публичный API python-docx не покрывает все свойства таблиц.

Рисунки и подписи

```
![Рисунок 1 — Контекстная схема взаимодействия](images/context-diagram.png){width=90%
```

Или для сохранения авторского смысла:

```
![Рисунок 1 — Архитектура программной системы](images/architecture.png)
```

В интерфейс стоит добавить переключатель:

Рисунки:

- ☒ Вписывать в рабочую ширину страницы
- ☐ Сохранять исходный размер

А также режим проверки: выделять ошибкой путь к отсутствующей картинке. Для Mermaid процесс может автоматически преобразовывать .mmd в PNG/SVG перед запуском Pandoc, чтобы на GitHub исходник оставался Mermaid, а в DOCX была готовая схема.

Блок требований и примечаний

Для ТЗ часто полезны стандартизированные смысловые блоки:

```
::: {.requirement}
Система должна обеспечивать хранение журнала событий не менее 12 месяцев.
:::

::: {.note}
Допускается уточнение состава интеграций на этапе технического проектирования.
:::

::: {.warning}
Перед вводом в эксплуатацию требуется провести приёмочные испытания.
:::
```

Это fenced Divs — расширение Pandoc Markdown. В DOCX через атрибут custom-style их можно сопоставлять с конкретными стилями Word, например Requirement, Note, Warning. Синтаксис fenced Div поддерживает блоки с атрибутами; Pandoc умеет применять пользовательские стили DOCX к блокам, тексту и таблицам.^[3] ^[1]

Для первого этапа я бы сделал более явный вариант:

```
::: {custom-style="Requirement"}
Требование: система должна обеспечивать ...
:::
```

А в `reference.docx` один раз создал абзацный стиль `Requirement`:

- Times New Roman, 12 pt;
- обычный текст;
- отступы, соответствующие методике;
- при необходимости тонкая рамка или акцентная полоса.

Таблицы разных типов

Со временем почти наверняка понадобятся разные таблицы:

```
::: {custom-style="RequirementsTable"}

| № | Требование | Критерий приёмки |
| ---: | --- | --- |
| 1 | Авторизация | Пользователь входит через SSO |
| 2 | Журналирование | Операции видны аудитору |

:::
```

И:

```
::: {custom-style="TermsTable"}

| Термин | Определение |
| --- | --- |
| Пользователь | Лицо, использующее систему |
| Администратор | Пользователь с правами настройки |

:::
```

Последние версии Pandoc позволяют задавать `custom-style` для блоков и таблиц, если стиль существует в `reference DOCX`. Однако из-за различий Word-стилей и генератора DOCX я бы всё равно заложил постобработку таблиц как обязательный «страховочный» слой. ^[4] ^[1]

Что добавить в интерфейс

Ваше окно можно сделать чуть богаче, но не перегружать.

Блок «Исходные данные»

Исходный Markdown:	[путь к файлу]	[Выбрать...]
Профиль оформления:	[ГОСТ 19 – НГУ, Times New Roman 12 ▼]	
Reference DOCX:	[путь к файлу]	[Выбрать...]
Папка результата:	[Рядом с исходным Markdown]	

У поля `reference DOCX` должен быть режим:

- ☒ Использовать reference DOCX из выбранного профиля
- ☐ Указать свой reference DOCX

Это исключит частую ошибку, когда случайно выбран старый шаблон с Arial 11.

Блок «Оформление»

- ☒ Принудительно привести текст к Times New Roman, 12 pt
- ☒ Принудительно оформить все таблицы
 - └ ширина: по доступной ширине страницы
 - └ границы: чёрные, 0,5 pt
 - └ шапка: жирная, по центру
 - └ запрещать разрыв строки шапки
- ☒ Обновить оглавление при открытии в Word
- ☐ Открыть DOCX после сборки

Надпись «принудительно» важна: reference DOCX — это предпочтение, а post-processing — гарантия.

Блок «Проверка перед сборкой»

Перед нажатием «Создать DOCX» программа может делать статический анализ:

Проверка	Пример сообщения
Не найдено изображение	images/context.png отсутствует
Mermaid-блок	«Нужно преобразовать Mermaid в PNG»
Нет заголовка H1	«В документе не найден раздел верхнего уровня»
Некорректная таблица	«Таблица в строке 125 имеет разное число ячеек»
Нет метаданных	«Не заполнено поле title»
Слишком длинная строка	«Проверьте таблицу: возможен выход текста за границы»
Вручную введённый номер	«Заголовок 1.2 . . . будет лучше нумеровать стилями Word»

Для вашего рабочего процесса с GitHub особенно ценно добавить кнопку «Проверить Markdown» отдельно от «Создать DOCX». Тогда перед коммитом можно быстро проверить документ, не формируя Word-файл.

Надёжная обработка DOCX

Я бы использовал два этапа:

Этап 1. Pandoc
Markdown → DOCX с reference.docx

Этап 2. Python post-processing

DOCX → выравнивание таблиц, границы, шрифты, защита шапок

В Python после конвертации программа должна:

1. Открыть готовый DOCX.
2. Установить базовые стили Normal, Heading 1, Heading 2, Heading 3 на Times New Roman 12.
3. Пройти по таблицам.
4. Задать для каждой:
 - ширину в пределах доступной области;
 - чёрные внешние и внутренние линии толщиной 0,5 pt;
 - Times New Roman 12 для всех run внутри ячеек;
 - полужирное начертание шапки;
 - запрет делить строку-шапку между страницами;
 - отключение автоизменения размеров по содержимому.
5. Настроить параметры секций: A4, поля.
6. Сохранить результат.

Это важнее, чем пытаться добиться идеальной передачи всех стилей из Google Docs в DOCX. Google Docs полезен для ручного редактирования, но для стабильного выпуска ТЗ я бы считал `reference.docx` вспомогательным слоем, а кодovou обработку — источником гарантированных требований.

Реализация по этапам

Этап 1: устойчивый конвертер

Доработать текущий GUI:

- выбор `.md`;
- выбор `reference.docx`;
- создание `.docx` рядом с `.md`;
- сохранение последнего выбранного шаблона;
- кнопка «Открыть папку результата»;
- понятный журнал Pandoc и ошибок;
- проверка наличия `pandoc`.

Этап 2: профиль ГОСТ 19

Добавить встроенный профиль:

ГОСТ 19 / НГУ:

- Times New Roman, 12 pt;
- A4;
- поля по вашей методике;
- основной текст по ширине;
- таблицы: 100%, линии 0,5 pt;
- Heading 1-3;
- колонтитулы;
- номера страниц.

Настройки профиля лучше хранить в JSON, а не жёстко в Python. Тогда вы сможете исправить толщину таблицы, не меняя код:

```
{
  "name": "ГОСТ 19 – НГУ",
  "font": "Times New Roman",
  "font_size_pt": 12,
  "table_border_pt": 0.5,
  "table_width_percent": 100,
  "table_header_bold": true,
  "table_header_center": true
}
```

Этап 3: Markdown-валидатор

Добавить:

- проверку изображений;
- предупреждение по Mermaid;
- проверку структуры H1 → H2 → H3;
- проверку метаданных;
- отчёт в интерфейсе до сборки.

Этап 4: Mermaid и сборка из GitHub

Добавить обработку Mermaid:

```
diagram.mmd / Mermaid-блок
↓
mermaid-cli
↓
PNG или SVG
↓
Pandoc
```


↓
DOCX

В перспективе можно сделать кнопку:

[Собрать из локального репозитория]

Она будет выполнять:

```
git pull
→ проверить Markdown
→ сгенерировать Mermaid-схемы
→ собрать DOCX
→ открыть результат
```

Но это стоит делать только после того, как базовый выпуск DOCX станет стабильным.

Практический выбор

Я бы сейчас выбрал такой минимальный, но сильный вариант:

1. Сохранить текущий GUI и работу через Pandoc.
2. Встроить единственный профиль «ГОСТ 19 — Times New Roman 12».
3. Добавить post-processing через Python для всех таблиц: ширина, чёрные границы 0,5 pt, Times New Roman 12.
4. Добавить в Markdown только четыре стандартных элемента:
 - YAML-метаданные;
 - заголовки #, ##, ###;
 - таблицы;
 - изображения / Mermaid с предварительным рендером.
5. После стабильной проверки — ввести fenced Divs для «Требования», «Примечания», «Предупреждения» и отдельные классы таблиц.

Такой подход позволит вам не менять привычный GitHub Markdown-рендерер, не превращать ТЗ в сложный Word-файл до последнего этапа и при этом получать стандартизированный DOCX для сдачи.

✱

1. <https://pandoc.org/demo/example33/16.1-output.html>
2. <https://pandoc.org/demo/example33/3.4-options-affecting-specific-writers.html>
3. <https://pandoc.org/demo/example33/8.18-divs-and-spans.html>
4. <https://pandoc.org/demo/example33/7.6-other-extensions.html>
5. <https://git.pashev.ru/pandoc/commit/doc/lua-filters.md?id=9dd4d077c0729c200a5172dbeaee92f8a0d985f5>

6. `work.documentation.markdown_github`
7. `preferences.document_formatting`
8. <https://pandoc.org/MANUAL.html>
9. <https://pandoc.org/demo/UsersGuide.docx>
10. https://www.uv.es/wiki/pandoc_manual_2.7.3.wiki?175
11. <https://pandoc.org/demo/example33/16.2-input.html>
12. <https://stackoverflow.com/questions/17858598/add-styling-rules-in-pandoc-tables-for-odt-docx-output-table-borders>
13. <https://pandoc.org/lua-filters.html>
14. <https://stackoverflow.com/questions/63536579/different-table-styles-in-same-source-document>
15. <https://pandoc.org/filters.html>
16. <https://github.com/jgm/pandoc/issues/4139>
17. <https://tex.stackexchange.com/questions/525924/with-pandoc-how-to-apply-a-style-to-a-fenced-div-block>